Literature Review: Deep Learning for Source Code Generation

Prepared to accompany discussion of: Alshehri, W., Kammoun Jarraya, S., Allinjawi, A. (2025). New Deep Learning-Based Approach for Source Code Generation: Application to Computer Vision Systems. AI, 6(7), 162.

Overview

The base paper proposes a grammar-based CNN encoder–decoder combined with a tree-based (AST) representation for source code generation, evaluated on the HEARTHSTONE benchmark and a newly created computer-vision-specific dataset, AST2CVCode, along with a new evaluation metric, BLEU+, designed to better capture functional correctness. This document summarizes recent related work (2024–2025) that situates this paper within the broader field: surveys tracing the shift from RNN/CNN-based models to LLMs, benchmarks used to evaluate modern code-generation systems, domain-specific structured generation work closest in spirit to the base paper, and the shift toward reinforcement-learning-based training.

1. Surveys

Deep Learning for Code Generation: A Survey (Zhang et al., Science China Information Sciences, 2024)

https://link.springer.com/article/10.1007/s11432-023-3956-3

Formalizes the code-generation pipeline and organizes existing solutions by architecture, model-agnostic enhancement strategy, evaluation metric, and task type. Outlines open challenges facing large code models and proposes future research directions. Useful as a taxonomy reference for classifying where the base paper's CNN/AST approach sits relative to sequence-based and Transformer-based methods.

Code Generation with Large Language Models: A Survey (Applied Intelligence, 2025/2026)

https://link.springer.com/article/10.1007/s10489-026-07230-0

Traces the evolution of code generation from classical program synthesis through pre-Transformer neural approaches to today's large-scale LLMs, including a timeline of major model releases from 2020–2025. Discusses how task definitions, methods, and evaluation protocols have co-evolved. Best single source for framing '2024–2025 in this field' for a presentation.

A Survey on Large Language Models for Software Engineering (Science China Information Sciences, 2025/2026)

https://link.springer.com/article/10.1007/s11432-025-4670-0

Positions code generation as one task within the wider space of LLM-based software engineering, alongside testing, program repair, and multi-agent development systems. Reflects the field's broader shift from single-shot code synthesis toward LLM agents performing end-to-end software engineering workflows.

2. Benchmarks

SWE-bench (Jimenez et al., 2023) and its 2024–2025 successors

https://arxiv.org/pdf/2505.20411 (SWE-rebench)

Introduced execution-based, repository-level coding tasks sourced from real GitHub issues, a far more realistic evaluation setting than isolated function generation. Because SWE-bench predates many newer LLMs, 2024–2025 follow-ups such as SWE-bench+, Multi-SWE-bench, SWE-bench Pro, SWE-Gym, and SWE-rebench address contamination, multilingual coverage, and continuous decontaminated evaluation.

LiveCodeBench (Jain et al., 2024)

https://artificialanalysis.ai/evaluations/livecodebench

Continuously collects new competitive-programming problems from LeetCode, AtCoder, and CodeForces to remain contamination-free, and evaluates a broader range of coding capabilities beyond generation — including self-repair, code execution, and test-output prediction.

BigCodeBench (Zhuo et al., ICLR 2025)

https://arxiv.org/abs/2406.15877

Contains 1,140 realistic Python programming tasks requiring 723 function calls across 139 libraries and 7 domains, built through a year-long human–LLM collaborative annotation process. Even top LLMs solve only about 60% of the docstring-based split and 50% of the natural-language-instruction split, versus 97% human performance, showing substantial remaining headroom in practical code generation.

SWE-rebench (2025)

https://arxiv.org/pdf/2505.20411

An automated pipeline for continuously collecting fresh software-engineering tasks and running decontaminated evaluation, directly addressing the concern that older benchmarks like the original SWE-bench are becoming compromised by training-data memorization rather than genuine problem-solving.

3. Domain-Specific / Structured Generation (closest analogs to the base paper)

Code Generation System Based on MDA and Convolutional Neural Networks (Frontiers in AI, 2025)

https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2025.1491958/full

Automates source code generation from architectural (Model-Driven Architecture) diagrams using CNNs, aiming to reduce development time by connecting planning, structuring, and development stages. Structurally the closest 2025 analog to the base paper's philosophy of constraining CNN-based generation with structured, non-free-text input.

TuRTLe: A Unified Evaluation of LLMs for RTL Generation (2025)

https://arxiv.org/pdf/2504.01986

Evaluates LLMs on hardware-description-language (Verilog/RTL) code generation, a domain — like the base paper's computer-vision code target — where general-purpose LLMs underperform and structurally-aware, domain-specific approaches still provide meaningful advantages.

4. Training Paradigm Shift: Reinforcement Learning and Execution Feedback

A Survey of Reinforcement Learning for Software Engineering (2025)

https://arxiv.org/pdf/2507.12483

Reviews the shift from supervised token-prediction training (the paradigm used by the base paper's CNN model) toward reinforcement learning, where models are rewarded for producing code that actually executes correctly and passes tests, rather than for matching reference tokens.

DeepSeekMath / DeepSeek-R1 lineage and derivative code models (2024–2025)

https://arxiv.org/pdf/2607.07748 (references DeepCoder, CodeV-R1)

RL-for-reasoning training recipes originally developed for mathematical reasoning were rapidly adapted into code models (e.g., DeepCoder, CodeV-R1) that reason step-by-step and self-verify before emitting final code — a capability qualitatively different from the single-pass grammar-rule prediction used in the base paper.

Summary Table

Base paper (2025): grammar-based CNN + AST + attention, evaluated on HEARTHSTONE (BLEU 81.4, EM 30.3, Acc+ 62.1%) and its own AST2CVCode dataset (BLEU 86.2, EM 51.9%, BLEU+ 92.0). Field trend (2024–2025): move to LLM-based, execution-evaluated, agentic, RL-trained code generation, with domain-specific structured

approaches like the base paper remaining a relevant niche where correctness guarantees and interpretability matter more than raw scale.